

EE-2003

Computer Organization and Assembly Language

Week 05 – Lecture 09

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Register + Offset Addressing)
- 3 CMP Instruction Basics
- 4 Conditional Jumps
- 5 Unconditional Jump (JMP)
- 6 Relative Addressing
- 7 Types of Jump

Quick Review: What is Done in Last Week?

- Register Indirect Addressing
- Loops & Program Control in Assembly
- Register with Offset Addressing
- Segment Association & Address Wraparound

Example: Adding Ten Numbers

Assembly Program (Register + Offset Addressing)

```
; a program to add ten numbers using register + offset addressing
[org 0x0100]

        mov     bx, 0           ; initialize array index to zero
        mov     cx, 10          ; load count of numbers in cx
        mov     ax, 0           ; initialize sum to zero
11:      add     ax, [num1+bx]    ; add number to ax
        add     bx, 2           ; advance bx to next index
        sub     cx, 1           ; numbers to be added reduced
        jnz     11             ; if numbers remain add next
        mov     [total], ax     ; write back sum in memory
        mov     ax, 0x4c00      ; terminate program
        int     0x21

num1:    dw      10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total:   dw      0
```

Explanation:

- BX used as an index with offset addressing (num1+bx).
- CX holds the loop counter (10 numbers).
- AX accumulates the sum of all numbers.
- Result stored in memory at label total.

Example: Adding Numbers with Byte Variables

Assembly Program (Using Register + Offset Addressing)

```
; a program to add numbers using byte variables
[org 0x0100]

    ; initialize stuff
    mov ax, 0           ; reset the accumulator
    mov cx, 10          ; set the iteration count
    mov bx, 0           ; base index = 0 (first element)

outerloop:
    add ax, [num1 + bx] ; add element at offset BX
    add bx, 2           ; move BX to next element (word size = 2)

    sub cx, 1           ; decrement counter
    jnz outerloop       ; repeat until CX = 0

    mov [result], ax    ; store final sum

    mov ax, 0x4c00      ; terminate program
    int 0x21

    num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
    result: dw 0
```

Explanation of Program

Key Points:

- BX is initialized to 0 $\rightarrow$ ensures the loop starts from the first element at `num1+0`.
- Each iteration: `add bx, 2` advances BX to the next element (since each word = 2 bytes).
- CX acts as the loop counter (10 iterations in this case).
- AX accumulates the running sum of all numbers.
- Final sum is stored in memory location `result`.

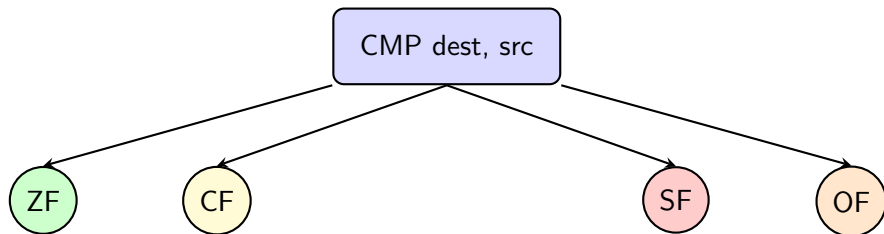
Why BX = 0 initially?

- To correctly access the first number in the array (`num1`).
- Without resetting BX, the program might skip or fetch the wrong memory location.

CMP Instruction Basics

CMP Instruction (Refer Page 41: Chapter 3 (First Table)):

- CMP dest, src performs $DEST - SRC$
- Updates **flags** (ZF, CF, SF, OF) but does not change operands
- Enables **conditional jumps**



CMP subtracts internally, then sets flags for later decision making.

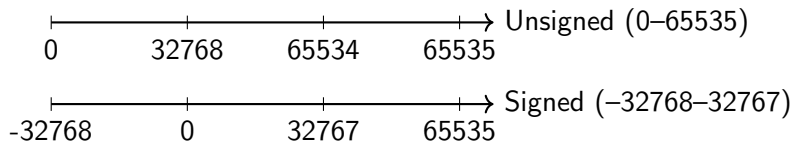
Signed vs Unsigned Interpretation

Unsigned Numbers

- Range: 0 ... 65535
- Jumps: JA (above), JB (below)
- Example: $65534 > 2$

Signed Numbers

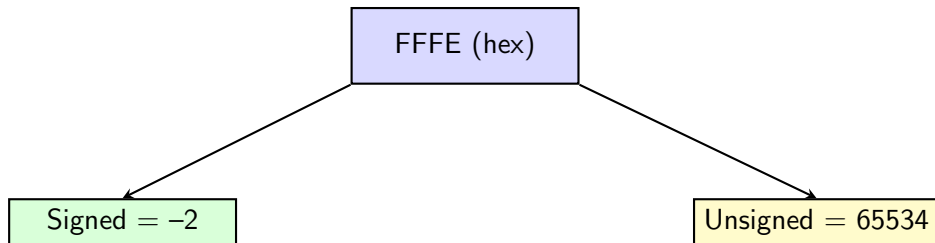
- Range: $-32768 \dots +32767$
- Jumps: JG (greater), JL (less)
- Example: $-2 < 2$



Same bit pattern interpreted differently as signed vs unsigned.

Two's Complement Example

- Signed numbers stored in **two's complement**.
- Example calculation for -2 (16-bit):
 - 1 $+2 = 0002h \rightarrow 0000\ 0000\ 0000\ 0010$
 - 2 Invert bits $\rightarrow 1111\ 1111\ 1111\ 1101$
 - 3 Add 1 $\rightarrow 1111\ 1111\ 1111\ 1110 = FFFEh$
- Same binary may mean:
 - -2 (signed) OR 65534 (unsigned)



Effect on jumps: JA (unsigned) $\rightarrow$ jump taken, JG (signed) $\rightarrow$ jump not taken.

Flags after CMP: Unsigned vs Signed

Unsigned Comparisons

Condition	Flags
DEST = SRC	ZF = 1
DEST < SRC	CF = 1
DEST ≤ SRC	ZF = 1 OR CF = 1
DEST ≥ SRC	CF = 0
DEST > SRC	ZF = 0 AND CF = 0

Signed Comparisons

Condition	Flags
DEST < SRC	SF ≠ OF
DEST ≤ SRC	ZF = 1 OR SF ≠ OF
DEST ≥ SRC	SF = OF
DEST > SRC	ZF = 0 AND SF = OF

Key Point: CMP does subtraction internally, but:

- For **unsigned**, result is interpreted with CF/ZF.
- For **signed**, result is interpreted with SF/OF.
- Same binary number can mean different things depending on signed/unsigned.

CMP Instruction Flow

- CMP subtracts SRC from DEST, but keeps values unchanged.
- Flags are set based on the result.
- Conditional jumps check these flags to decide program flow.

Conditional Jumps: Overview

- Conditional jumps depend on CPU **flags** (CF, ZF, SF, OF, PF).
- Intel provides multiple mnemonics for readability:
 - JZ $\equiv$ JE (jump if zero / equal)
 - JNZ $\equiv$ JNE (jump if not zero / not equal)
- Same opcode, different names $\rightarrow$ helps logical clarity in assembly programs.

Conditional Jumps: Carry Flag (CF)

Instruction	Condition
JC / JB / JNAE	Jump if carry, below, not above/equal ($CF = 1$)
JNC / JNB / JAE	Jump if not carry, not below, above/equal ($CF = 0$)

Example: After `CMP dest, src` →

- $CF = 1$ if unsigned $src > dest$.
- $CF = 0$ if unsigned $src \leq dest$.

Conditional Jumps: Zero Flag (ZF)

Instruction	Condition
JE / JZ	Jump if equal / zero ($ZF = 1$)
JNE / JNZ	Jump if not equal / not zero ($ZF = 0$)

Use case:

- Compare two operands with CMP.
- If result is 0 $\rightarrow ZF = 1 \rightarrow$ JE is taken.
- Else $\rightarrow ZF = 0 \rightarrow$ JNE is taken.

Unsigned Comparisons

Instruction	Condition
JA / JNBE	Jump if above (unsigned) ($ZF = 0 \wedge CF = 0$)
JNA / JBE	Jump if not above / below or equal ($ZF = 1 \vee CF = 1$)

Note: Operands are treated as **unsigned** numbers.

Signed Comparisons

Instruction	Condition
JL / JNGE	Jump if less ($SF \neq OF$)
JNL / JGE	Jump if not less / greater or equal ($SF = OF$)
JG / JNLE	Jump if greater ($ZF = 0 \wedge SF = OF$)
JNG / JLE	Jump if not greater / less or equal ($ZF = 1 \vee SF \neq OF$)

Note: Operands interpreted as **signed** numbers.

Other Flag-Based Jumps

Instruction	Flag	Condition
J0, JN0	OF	Jump if overflow / not overflow
JS, JNS	SF	Jump if sign / not sign
JP(JPE), JNP(JPO)	PF	Jump if parity even / odd

Special Case: JCXZ

- JCXZ $\rightarrow$ Jump if $CX = 0$.
- Independent of any flag.
- Special treatment because CX often serves as a loop counter.

Example Program: Adding 10 Numbers

```
[org 0x0100]
    mov  bx, 0      ; initialize array index
    mov  ax, 0      ; initialize sum

l1:  add  ax, [num1+bx] ; add number
     add  bx, 2      ; next index
     cmp  bx, 20     ; check if end
     jne  l1         ; if not, continue

     mov  [total], ax ; store result
     mov  ax, 0x4c00  ; exit
     int  0x21

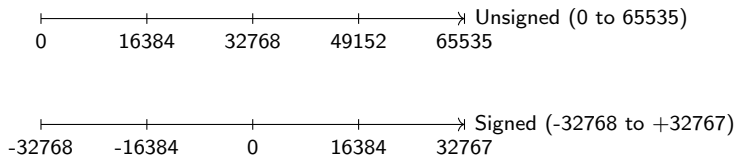
num1: dw 10,20,30,40,50,10,20,30,40,50
total: dw 0
```

- JNE and JNZ share same opcode.
- Debugger may display only one mnemonic.
- Jump operand stored as relative offset (e.g., F2) instead of full address.
- Short relative jumps: efficient encoding.

Conditional Jumps: Visual Summary

Flag	Condition	Jumps
ZF	Zero / Equality	JE, JZ, JNE, JNZ
CF	Carry / Below	JC, JB, JNC, JAE
SF, OF	Sign / Overflow	JL, JG, JGE, JLE
PF	Parity	JP, JPE, JNP, JPO

Signed vs Unsigned Number Line



- **Unsigned jumps:** JA, JB → MSB as data bit.
- **Signed jumps:** JL, JG → MSB as sign bit.

Unconditional Jump (JMP)

- Till now, data was placed at the **end of code**.
- No restriction: data can be placed **anywhere**.
- If data is placed at 0100h (start of program):
 - Debugger expects code at 0100h.
 - It misinterprets data as instructions.
- Solution: Use **Unconditional Jump (JMP)** as the first instruction.
- JMP transfers control unconditionally, independent of flags.

Example 3.2: Adding 10 Numbers

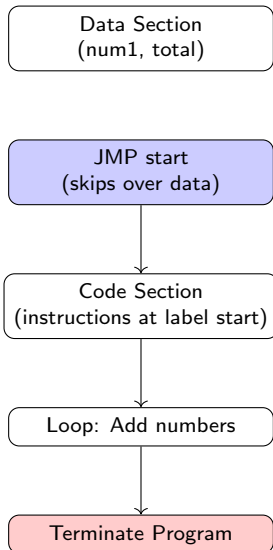
Assembly Program

```
[org 0x0100]
    jmp start                ; unconditionally jump over data

num1:  dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0

start: mov bx, 0             ; initialize array index
      mov ax, 0             ; initialize sum
l1:    add ax, [num1+bx]     ; add number to AX
      add bx, 2             ; next element
      cmp bx, 20            ; done?
      jne l1               ; repeat if not
      mov [total], ax       ; save result
      mov ax, 0x4c00        ; terminate
      int 0x21
```

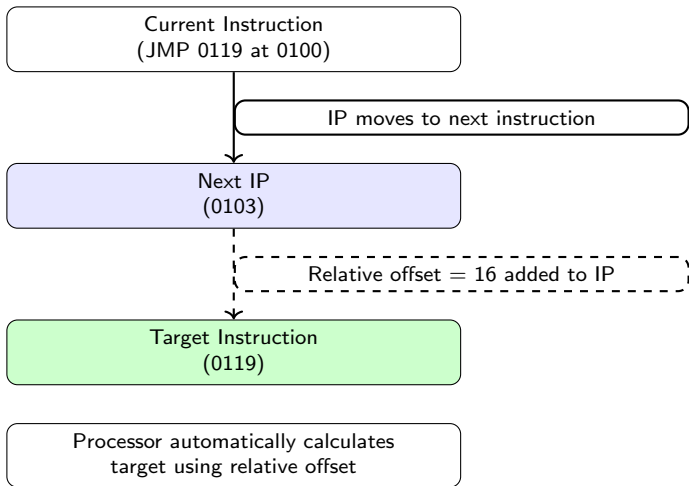
Execution Flow of Unconditional Jump



Relative Addressing: Concept

- In assembly, an instruction like `JMP 0119` is unconditional.
- The debugger shows opcode: `F2 16 00` (`F2` = opcode, `1600` = operand in little-endian word order).
- This is ****position relative addressing****, not absolute:
 - Operand tells how much to move ****forward or backward**** from the current IP (Instruction Pointer).
 - At execution, IP points to the next instruction after the current one.
 - Adding the operand to IP gives the target location (here, `0119`).
- This allows labels in code without needing exact memory addresses.

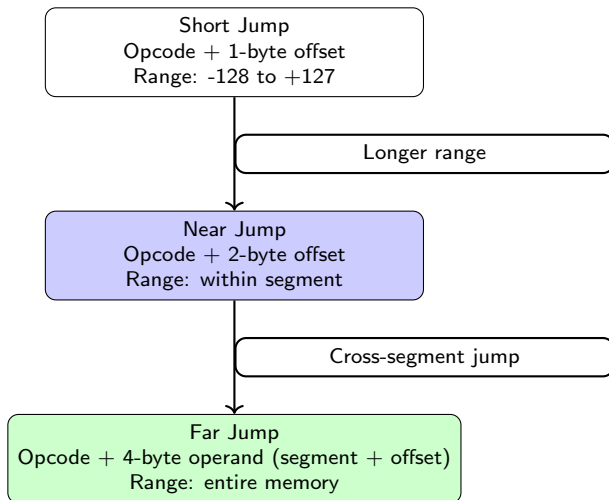
Relative Addressing: Visual Representation



Types of Jump: Overview

- Jumps differ in ****instruction size**** and ****range of memory****.
- ****Short Jump****
 - Opcode + 1-byte operand
 - Relative offset: -128 to +127 bytes
 - Conditional jumps can only be short
- ****Near Jump****
 - Opcode + 2-byte offset
 - Can jump ****anywhere within the current segment****
 - Relative addressing with 16-bit offset
- ****Far Jump****
 - Opcode + 4-byte operand (2-byte segment + 2-byte offset)
 - Absolute jump: can go to ****anywhere in memory****
 - Loads CS with segment, IP with offset
- Instruction size and range: **Short = 2 bytes, Near = 3 bytes, Far = 5 bytes**

Types of Jump: Visual Representation



Thank You!

Any Questions?

- **We covered:** addressing modes (register indirect, register+offset), loop control with CX, CMP and flags (ZF, CF, SF, OF), signed vs. unsigned interpretation, two's complement, conditional & unconditional jumps, and relative / short / near / far jumps.
- **Big idea:** CMP performs a subtraction to set flags; the *same bits* can mean different things for signed vs. unsigned, so choose the matching jump family.

“Master addressing and flags, and you master assembly programming.”